

Java Coding Interview Questions & Solutions

4 Essential Topics · 2 Solutions Each · With Output

#	Topic	Best Approach
1	Fibonacci Sequence	Iterative
2	Palindrome Check	Two Pointers
3	Factorial	Iterative
4	Prime Number Check	Square Root Optimized

Q1: Fibonacci Sequence

Solution 1: Iterative

```
public class Fibonacci {  
    public static int fibonacci(int n) {  
        if (n <= 1) return n;  
        int a = 0, b = 1;  
        for (int i = 2; i <= n; i++) {  
            int temp = a + b;  
            a = b;  
            b = temp;  
        }  
        return b;  
    }  
    public static void main(String[] args) {  
        int n = 10;  
        System.out.println("Fibonacci of " + n + " = " + fibonacci(n));  
        System.out.print("Series: ");  
        for (int i = 0; i <= n; i++) System.out.print(fibonacci(i) + " ");  
    }  
}
```

Output:

```
Fibonacci of 10 = 55  
Series: 0 1 1 2 3 5 8 13 21 34 55
```

Solution 2: Recursive

```

public class Fibonacci {
    public static int fibonacci(int n) {
        if (n <= 1) return n;
        return fibonacci(n - 1) + fibonacci(n - 2);
    }
    public static void main(String[] args) {
        int n = 10;
        System.out.println("Fibonacci of " + n + " = " + fibonacci(n));
        System.out.print("Series: ");
        for (int i = 0; i <= n; i++) System.out.print(fibonacci(i) + " ");
    }
}

```

Output:

```

Fibonacci of 10 = 55
Series: 0 1 1 2 3 5 8 13 21 34 55

```

Comparison:

	Iterative	Recursive
Time Complexity	$O(n)$	$O(2^n)$
Space Complexity	$O(1)$	$O(n)$ call stack
Best for	Large values of n	Small values of n

Q2: Palindrome Check

Solution 1: Two Pointers

```

public class Palindrome {
    public static boolean isPalindrome(String str) {
        int left = 0, right = str.length() - 1;
        while (left < right) {
            if (str.charAt(left) != str.charAt(right)) return false;
            left++;
            right--;
        }
        return true;
    }
    public static void main(String[] args) {
        System.out.println("racecar is palindrome: " + isPalindrome("racecar"));
        System.out.println("hello is palindrome: " + isPalindrome("hello"));
    }
}

```

Output:

```

racecar is palindrome: true
hello is palindrome: false

```

Solution 2: StringBuilder reverse()

```
public class Palindrome {
    public static boolean isPalindrome(String str) {
        String reversed = new StringBuilder(str).reverse().toString();
        return str.equals(reversed);
    }
    public static void main(String[] args) {
        System.out.println("madam is palindrome: " + isPalindrome("madam"));
        System.out.println("java is palindrome: " + isPalindrome("java"));
    }
}
```

Output:

```
madam is palindrome: true
java is palindrome: false
```

Comparison:

	Two Pointers	StringBuilder
Time Complexity	O(n)	O(n)
Space Complexity	O(1)	O(n)
Best for	Memory efficiency	Quick & simple code

Q3: Factorial

Solution 1: Iterative

```
public class Factorial {
    public static long factorial(int n) {
        long result = 1;
        for (int i = 2; i <= n; i++) result *= i;
        return result;
    }
    public static void main(String[] args) {
        System.out.println("Factorial of 10 = " + factorial(10));
        for (int i = 1; i <= 5; i++)
            System.out.println(i + "! = " + factorial(i));
    }
}
```

Output:

```
Factorial of 10 = 3628800
1! = 1
2! = 2
3! = 6
4! = 24
5! = 120
```

Solution 2: Recursive

```
public class Factorial {
    public static long factorial(int n) {
        if (n == 0 || n == 1) return 1;
        return n * factorial(n - 1);
    }
    public static void main(String[] args) {
        System.out.println("Factorial of 10 = " + factorial(10));
        for (int i = 1; i <= 5; i++)
            System.out.println(i + "! = " + factorial(i));
    }
}
```

Output:

```
Factorial of 10 = 3628800
1! = 1
2! = 2
3! = 6
4! = 24
5! = 120
```

Comparison:

	Iterative	Recursive
Time Complexity	$O(n)$	$O(n)$
Space Complexity	$O(1)$	$O(n)$ call stack
Risk	None	StackOverflow for large n
Best for	Large values of n	Small values of n

Q4: Prime Number Check

Solution 1: Basic Loop

```
public class PrimeNumber {
    public static boolean isPrime(int n) {
        if (n <= 1) return false;
        for (int i = 2; i < n; i++)
            if (n % i == 0) return false;
        return true;
    }
    public static void main(String[] args) {
        System.out.println("17 is prime: " + isPrime(17));
        System.out.print("Primes 1-50: ");
        for (int i = 1; i <= 50; i++)
            if (isPrime(i)) System.out.print(i + " ");
    }
}
```

```
}  
}
```

Output:

```
17 is prime: true  
Primes 1-50: 2 3 5 7 11 13 17 19 23 29 31 37 41 43 47
```

Solution 2: Optimized Square Root

```
public class PrimeNumber {  
    public static boolean isPrime(int n) {  
        if (n <= 1) return false;  
        if (n <= 3) return true;  
        if (n % 2 == 0 || n % 3 == 0) return false;  
        for (int i = 5; i * i <= n; i += 6)  
            if (n % i == 0 || n % (i + 2) == 0) return false;  
        return true;  
    }  
    public static void main(String[] args) {  
        System.out.println("17 is prime: " + isPrime(17));  
        System.out.print("Primes 1-50: ");  
        for (int i = 1; i <= 50; i++)  
            if (isPrime(i)) System.out.print(i + " ");  
    }  
}
```

Output:

```
17 is prime: true  
Primes 1-50: 2 3 5 7 11 13 17 19 23 29 31 37 41 43 47
```

Comparison:

	Basic Loop	Square Root
Time Complexity	$O(n)$	$O(\sqrt{n})$
Space Complexity	$O(1)$	$O(1)$
Best for	Small numbers	Large numbers

Summary — Best Approaches

#	Question	Best Approach	Why
1	Fibonacci	Iterative	$O(n)$ time, $O(1)$ space
2	Palindrome	Two Pointers	No extra space needed

3	Factorial	Iterative	Avoids stack overflow
4	Prime Check	Square Root	Much faster for large n